CORE103L Computational Thinking
LW9 - Nested Lists



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
- 1.1 Instructions . . . 2
- 1.2 Marking scheme . . . 2
  - 1.2.1 Lab Objectives . . . 2
- 1.3 Late submission policy . . . 2
- 1.4 Use of AI . . . 3
- 1.5 Viva . . . 3

2 Prerequisites **4**
- 2.1 Assert Statements . . . 4
- 2.2 Pytest . . . 4

3 Lab Exercises **6**
- 3.1 Crossword Positions . . . 6
- 3.2 Crowd Check . . . 8
- 3.3 Study Hours . . . 10
- 3.4 Color Neighbors . . . 12
- 3.5 Combo Deals . . . 14
- 3.6 Solo Invigilators . . . 16

Introduction

1.1 Instructions

- This lab will contribute 1% towards your final grade.
- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *LW9_aa123456.zip* where *aa123456* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise of both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 10.

1.2.1 Lab Objectives

Following are the lab objectives of this lab:

- Understand the concept of nested lists in Python and their role in organizing complex data structures.
- Explore techniques for accessing and manipulating elements within nested lists.
- Apply nested lists to solve programming challenges that involve multi-dimensional data, demonstrating proficiency in nested list manipulation and problem-solving.

1.3 Late submission policy

There is no late submission policy for the lab.

1.4 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.5 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
    assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, "x should be a non-negative number"
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
    pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
    pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
    pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
============================= test session starts ========
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py ..........

============================== 10 passed in 0.06s =========
```

However, if all the test fail, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
============================= test session starts =============================
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py FFFFFFFFFF

================================== FAILURES ===================================
_____________________________ test_question1[lst10-lst20-e54dc15ccafa608142ffa6340b035c327f972e24882052cfc3345f240f4ee237] ___
```

Lab Exercises

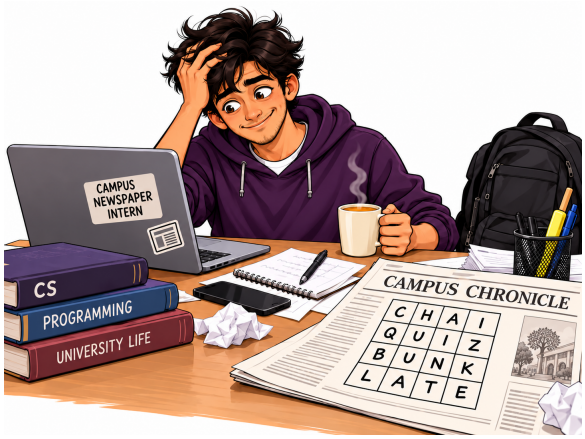
3.1 Crossword Positions

This question should be implemented in the file ***q1.py****.*

Problem Description

The campus newspaper is preparing a word grid containing four campus-related words:

C	H	A	I
Q	U	I	Z
B	U	N	K
L	A	T	E



An intern stored the grid as a single string, with rows separated by the newline character `'\n'`:

```
'CHAI\nQUIZ\nBUNK\nLATE'
```

The layout software needs the position of each letter in the format **[letter, row, column]**.

Function Description

Write a function **crossword_positions** that takes crossword, a string representing the grid, and **returns** a list of lists. Each inner list contains a letter, its row index, and its column index.

Rows and columns are indexed from **0**, starting at the top-left corner. The output must be ordered row by row, from left to right.

Note

You may assume that:

- crossword is a non-empty string.
- Every row has the same positive number of letters.
- Every character is an English letter (uppercase or lowercase) or a newline.
- The string does not begin or end with a newline, and there are no empty rows.
- Newline characters are not included in the output.

Sample Interaction

```
>>> crossword_positions('CHAI\nQUIZ\nBUNK\nLATE')
[['C', 0, 0], ['H', 0, 1], ['A', 0, 2], ['I', 0, 3],
 ['Q', 1, 0], ['U', 1, 1], ['I', 1, 2], ['Z', 1, 3],
 ['B', 2, 0], ['U', 2, 1], ['N', 2, 2], ['K', 2, 3],
 ['L', 3, 0], ['A', 3, 1], ['T', 3, 2], ['E', 3, 3]]

>>> crossword_positions('DS\nAL\nGO')
[['D', 0, 0], ['S', 0, 1], ['A', 1, 0],
 ['L', 1, 1], ['G', 2, 0], ['O', 2, 1]]

>>> crossword_positions('CHAI')
[['C', 0, 0], ['H', 0, 1], ['A', 0, 2], ['I', 0, 3]]
```

Sample Interaction Explanation

In the first example, each row is processed from left to right. Row indices increase from **0**, while column indices restart at **0** for each new row.

Row	Text	Entries
0	CHAI	['C', 0, 0] … ['I', 0, 3]
1	QUIZ	['Q', 1, 0] … ['Z', 1, 3]
2	BUNK	['B', 2, 0] … ['K', 2, 3]
3	LATE	['L', 3, 0] … ['E', 3, 3]

The second example has 3 rows and 2 columns, showing that the grid need not be square.

The third example contains only one row.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q1.py
```

3.2 Crowd Check

This question should be implemented in the file ***q2.py****.*

Problem Description

The Wellness Peers are hosting *The Reset*, a lunchtime event in the campus amphitheatre. The seating area is divided into an $n \times n$ grid of sections.

Each value represents the **net change** in students in that section (students who joined minus students who left). A negative value means more students left than joined.



To compare attendance changes, the organizers monitor two routes across the seating area:

- **Primary diagonal:** From the top-left to the bottom-right.
- **Secondary diagonal:** From the top-right to the bottom-left.

They want to find how much the total net change differs between these two routes, regardless of which is greater.

Your task is to calculate the **absolute difference** between the sums of the two diagonals.

For example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 9 & 8 & 9 \end{bmatrix}$$

Primary diagonal	$1 + 5 + 9 = 15$
Secondary diagonal	$3 + 5 + 9 = 17$
Difference	$\lvert 15 - 17 \rvert = 2$

Function Description

Write a function **diagonal_difference** that takes net_changes, an $n \times n$ nested list of integers, and **returns** the absolute difference between the sums of its two diagonals.

Constraints

- $1 \leq n \leq 100$
- $-100 \leq$ `net_changes[i][j]` ≤ 100
- The function must not modify the original net_changes nested list.

Sample Interaction

```
>>> diagonal_difference([[11, 2, 4], [4, 5, 6], [10, 8, -12]])
15

>>> diagonal_difference([[1, 2], [3, 10]])
6

>>> diagonal_difference([[-5]])
0
```

Sample Interaction Explanation

In the first example, the net changes along the two diagonal routes are:

$$\begin{bmatrix} \mathbf{11} & 2 & \mathbf{4} \\ 4 & \mathbf{5} & 6 \\ \mathbf{10} & 8 & \mathbf{-12} \end{bmatrix}$$

Primary diagonal	$11 + 5 - 12 = 4$
Secondary diagonal	$4 + 5 + 10 = 19$
Difference	$\lvert 4 - 19 \rvert = 15$

The centre value 5 (purple) belongs to both diagonals, so it is counted in both sums. The difference of 15 shows how much the total net changes along the two routes differ.

In the second example, the diagonal sums are $1 + 10 = 11$ and $2 + 3 = 5$. Their absolute difference is 6.

In the third example, both diagonals contain the same value, -5, so the result is 0.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q2.py
```

3.3 Study Hours

This question should be implemented in the file ***q3.py****.*

Problem Description

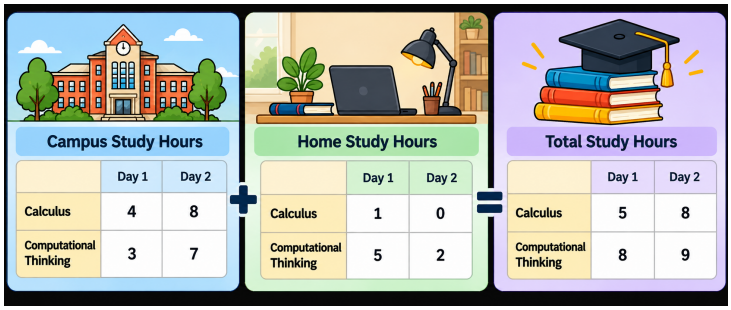
You record your study hours in two grids: one for studying on campus and another for studying at home. Each row represents a course, and each column represents a day.

For example, for Calculus and Computational Thinking over two days:

```
[[4, 8], [3, 7]]
```

This represents 4 and 8 hours of Calculus on days 1 and 2, and 3 and 7 hours of Computational Thinking.

Your task is to combine both grids to find the total study hours for each course on each day. The grids can only be combined if they have the same dimensions.



Function Description

Write a function **add_study_hours** that takes two nested lists of integers, campus_hours and home_hours.

If both grids have the same number of rows and columns, **return** a new grid containing the sums of corresponding elements.

Otherwise, **return** the string `"Grids don't have the same dimensions."`.

Constraints

- Every row within a grid has the same length.
- $1 \leq$ number of rows (courses) ≤ 6
- $1 \leq$ number of columns (days) ≤ 7
- $0 \leq$ each entry ≤ 12
- The input grids must not be modified.

Sample Interaction

```
>>> add_study_hours([[4, 8], [3, 7]], [[1, 0], [5, 2]])
[[5, 8], [8, 9]]

>>> add_study_hours([[5, 2], [0, 1], [1, 9]], [[2, 3], [4, 1], [0,
   2]])
[[7, 5], [4, 2], [1, 11]]

>>> add_study_hours([[1], [2]], [[3, 5], [4, 6]])
"Grids don't have the same dimensions."
```

Sample Interaction Explanation

In the first example, both grids are 2×2. Corresponding entries are added:

$$\begin{bmatrix} \mathbf{4} & 8 \\ 3 & \mathbf{7} \end{bmatrix} + \begin{bmatrix} \mathbf{1} & 0 \\ 5 & \mathbf{2} \end{bmatrix} = \begin{bmatrix} \mathbf{5} & 8 \\ 8 & \mathbf{9} \end{bmatrix}$$

Calculus on day 1 totals $4 + 1 = 5$ hours, while Computational Thinking on day 2 totals $7 + 2 = 9$ hours.

In the second example, both grids are 3×2, so the result has the same dimensions:

$$\begin{bmatrix} 5 & 2 \\ 0 & 1 \\ 1 & 9 \end{bmatrix} + \begin{bmatrix} 2 & 3 \\ 4 & 1 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 7 & 5 \\ 4 & 2 \\ 1 & 11 \end{bmatrix}$$

In the third example, campus_hours is 2×1, while home_hours is 2×2. Since their dimensions differ, the function returns `"Grids don't have the same dimensions."`.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q3.py
```

3.4 Color Neighbors

This question should be implemented in the file ***q4.py****.*

Problem Description

The Design Club is tiling a rectangular wall in which every tile has a unique colour. The first row in the list represents the top of the wall.



Given a colour, the club wants to find which colours touch it. The *neighbors* of a tile are the tiles directly above, below, left, and right of it. Diagonal tiles do not count. Missing directions at the edges of the wall are ignored.

Function Description

Write a function **color_neighbors** that takes:

- tile_wall: A nested list of strings representing the wall.
- target_color: A string representing the target tile's colour.

The function must **return** a list containing all neighbouring colours of target_color. **The order of colours in the returned list does not matter.**

Constraints

You may assume that:

- tile_wall has at least one row, and every row has the same non-zero number of elements.
- The number of rows and columns need not be equal.
- Every colour appears exactly once in tile_wall, and target_color is one of them.

Sample Interaction

```
>>> tile_wall = [
...     ["Red", "Blue", "Green", "Cyan"],
...     ["Yellow", "Black", "White", "Brown"],
...     ["Pink", "Orange", "Purple", "Gray"]
... ]

>>> color_neighbors(tile_wall, "Black")
['Blue', 'Orange', 'White', 'Yellow']

>>> color_neighbors(tile_wall, "Blue")
['Black', 'Green', 'Red']

>>> color_neighbors(tile_wall, "Gray")
['Brown', 'Purple']
```

(The outputs are shown in alphabetical order, but any order of the neighbouring colours is accepted.)

Sample Interaction Explanation

The target colour is shown in **purple** and its neighbours in **blue**.

Red	**Blue**	Green	Cyan
Yellow	**Black**	**White**	Brown
Pink	**Orange**	Purple	Gray

For the first sample, **Black** has four neighbours: `'Blue'`, `'Orange'`, `'White'` and `'Yellow'`.

Red	**Blue**	**Green**	Cyan
Yellow	**Black**	White	Brown
Pink	Orange	Purple	Gray

For the second sample, **Blue** is in the top row, so it has no top neighbour. Its neighbours are `'Black'`, `'Green'` and `'Red'`.

Red	Blue	Green	Cyan
Yellow	Black	White	**Brown**
Pink	Orange	**Purple**	**Gray**

For the third sample, **Gray** is in the bottom-right corner, so it has no bottom or right neighbour. Its neighbours are `'Brown'` and `'Purple'`.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q4.py
```

3.5 Combo Deals

This question should be implemented in the file ***q5.py****.*

Problem Description

The campus cafeteria, Cafeela, wants to introduce combo deals based on items students frequently order together. This is an example of *market basket analysis*.

The table below shows six orders placed on Monday. Each row represents one order, listing the items purchased together.

Order	Items Purchased Together
1	Chai, Samosa, Biscuit
2	Chai, Paratha, Omelette
3	Pizza Slice, Pepsi, Fries
4	Biryani, Raita, Lassi
5	Bun Kabab, Fries, Pepsi
6	Samosa, Roll, Jalebi

Function Description

Write a function **get_frequent_items** that takes:

- orders: A nested list of strings, where each inner list represents one order.
- target_item: A string representing the item to search for.

Consider only orders containing target_item. The function must **return** a list of all other items that appear most frequently in those orders.

If multiple items share the highest frequency, return all of them. **The order of items in the returned list does not matter.**

If no other item is ordered with target_item, or target_item does not appear in any order, return an empty list.

Note

You may assume that:

- Every order is non-empty and contains no repeated items.
- Item names are case-sensitive.

Sample Interaction

```
>>> monday_orders = [
...     ['Chai', 'Samosa', 'Biscuit'],
...     ['Chai', 'Paratha', 'Omelette'],
...     ['Pizza Slice', 'Pepsi', 'Fries'],
...     ['Biryani', 'Raita', 'Lassi'],
...     ['Bun Kabab', 'Fries', 'Pepsi'],
...     ['Samosa', 'Roll', 'Jalebi']]
>>> get_frequent_items(monday_orders, 'Pepsi')
['Fries']

>>> friday_orders = [
...     ['Chai', 'Samosa', 'Paratha', 'Cake Rusk'],
...     ['Chai', 'Biscuit', 'Samosa'],
...     ['Samosa', 'Cold Coffee', 'Biscuit'],
...     ['Pizza Slice', 'Fries', 'Pepsi', 'Brownie'],
...     ['Omelette', 'Paratha', 'Lassi', 'Samosa'],
...     ['Biscuit', 'Chai', 'Samosa', 'Pakora'],
...     ['Chai', 'Biscuit', 'Paratha']]
>>> get_frequent_items(friday_orders, 'Chai')
['Biscuit', 'Samosa']

>>> get_frequent_items(monday_orders, 'Brownie')
[]
```

(In the second example, `['Samosa', 'Biscuit']` is equally correct.)

Sample Interaction Explanation

First example: Pepsi appears in Monday orders 3 and 5. Fries accompanies Pepsi in both orders, while the other accompanying items appear only once. Therefore, the function returns `['Fries']`.



Second example: Chai appears in Friday orders 1, 2, 6 and 7. The frequencies of items ordered with Chai are:

Item	Orders with Chai	Frequency
Biscuit	2, 6, 7	3
Samosa	1, 2, 6	3
Paratha	1, 7	2
Cake Rusk	1	1
Pakora	6	1

Biscuit and Samosa tie for the highest frequency of 3, so both are returned: `['Biscuit', 'Samosa']`.

Third example: Brownie does not appear in any Monday order, so the function returns `[]`.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q5.py
```

3.6 Solo Invigilators

This question should be implemented in the file ***q6.py***.

Problem Description

For the Calculus exam, the university assigns invigilators to different positions in a large examination hall, represented as a grid.



Each position contains either:

- `'I'` – an invigilator is stationed here.
- `'E'` – no invigilator is stationed here.

An invigilator is considered *solo* if they are the only invigilator in their entire row **and** their entire column.

Your task is to count the number of solo invigilators.

Function Description

Implement the following functions:

- **count_solo_invigilators**: Takes hall_layout, a nested list of strings representing the examination hall, and **returns** the number of solo invigilators.
- **main**: Takes no parameters. Reads two space-separated integers representing the number of rows and columns, respectively.

  The next number of rows specified by the input each contain the specified number of space-separated **'I'** or **'E'** characters.

  The function constructs hall_layout, passes it to **count_solo_invigilators**, and **prints** the returned value.

Note

You may assume that:

- The number of rows and columns is positive.
- Every row contains exactly the specified number of columns.
- Each position contains either `'I'` or `'E'`.

Sample Interaction

In the samples below, lines written as **[Input -> value]** represent user input, not program output.

```
>>> main()
[Input -> 3 4]
[Input -> I E E E]
[Input -> E E I E]
[Input -> E I E E]
3
```

```
>>> main()
[Input -> 3 4]
[Input -> I I E E]
[Input -> E E I E]
[Input -> E E E I]
2
```

```
>>> main()
[Input -> 3 3]
[Input -> I I E]
[Input -> I I E]
[Input -> E E E]
0
```

Sample Interaction Explanation

In the first example, **main** reads **3 4**, indicating 3 rows and 4 columns. It then reads the next three lines and constructs hall_layout as:

```
[
    ['I', 'E', 'E', 'E'],
    ['E', 'E', 'I', 'E'],
    ['E', 'I', 'E', 'E']
]
```

The function **main** passes this nested list to **count_solo_invigilators**, which counts the solo invigilators and returns the result. Finally, **main** prints the returned value.

Solo invigilators are highlighted in **purple**.

I	E	E	E
E	E	**I**	E
E	**I**	E	E

In the first example, all three invigilators are alone in their respective rows and columns. Therefore, **count_solo_invigilators** returns **3**, which **main** prints.

I	I	E	E
E	E	**I**	E
E	E	E	**I**

In the second example, the first two invigilators share a row, while the remaining two are solo. Therefore, **count_solo_invigilators** returns **2**, which **main** prints.

I	I	E
I	I	E
E	E	E

In the third example, every invigilator shares a row or column with another. Therefore, **count_solo_invigilators** returns **0**, which **main** prints.

Testing

In order to test your function, type the following command on the terminal:

```
python -m pytest tests/test_q6.py
python3 -m pytest tests/test_q6.py
```